

PyTorch QAT Notes for an MNIST Linear Classifier

Updated guide covering inheritance, nn.Module, super().__init__(), STE, QAT basics, backend selection, and the meaning of the main preparation code.

Purpose. This note explains the ideas that usually cause confusion when first reading or writing QAT code in PyTorch, including why a model inherits from nn.Module, what observers do, why prepare_qat(...) is used, what backend = "x86" means, how STE fits into training, and why CUDA training is often followed by CPU conversion.

1. Inheritance in Python

1.1 Definition

Inheritance is an object-oriented programming concept in which a new class reuses the attributes and methods of an existing class. The new class is often called the child class or subclass, while the original class is called the parent class or base class.

The main benefit is that a developer can build on top of existing behavior instead of reimplementing the same logic from scratch.

1.2 Example

```
class SVC:
    def research(self):
        print("We research")

class CIAL(SVC):
    pass
```

In this example, CIAL inherits from SVC. Even though CIAL does not define its own research() method, it can still use that method because the method is inherited from the parent class.

1.3 Why Inheritance Is Useful

- It promotes code reuse.
- It reduces duplication.
- It makes code easier to organize.
- It allows customization by extending existing classes.
- A child class can keep the behavior of the parent class and still add new functionality of its own.

2. nn.Module in PyTorch

2.1 Why Models Inherit from nn.Module

In PyTorch, `nn.Module` is the base class for neural network models. When a custom model inherits from `nn.Module`, it becomes a proper PyTorch model class and can use the built-in infrastructure provided by PyTorch.

- Automatic registration of layers and parameters.
- Support for `model.parameters()`.
- Support for `model.state_dict()`.
- Support for `model.train()` and `model.eval()`.
- Support for moving the model to CPU or GPU with `model.to(device)`.

2.2 Why `super().__init__()` Is Needed

Inside the constructor, `super().__init__()` calls the constructor of the parent class. In this case, it runs the initialization logic of `nn.Module` before the child class adds its own layers.

This step is important because it initializes the internal structure that PyTorch uses to track submodules, parameters, buffers, and model-level behavior. Without it, PyTorch may not manage the model correctly.

```
class SimpleClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(784, 10)
```

3. STE in Quantization-Aware Training

3.1 What STE Means

STE stands for Straight-Through Estimator. It is a practical trick used when the forward pass includes non-differentiable operations such as rounding and clipping, which appear in quantization.

3.2 Why STE Is Needed

A true quantization step maps floating-point values to discrete integer levels. Operations such as `round(...)` have zero or undefined gradients at most points, so ordinary backpropagation would not work well if they were treated exactly.

STE solves this by using a split interpretation:

- Forward pass: simulate quantization effects such as rounding and clipping.
- Backward pass: approximate the gradient so that training can continue, often by treating the quantization step like an identity function.

3.3 Relation to QAT

In practice, QAT uses fake quantization in the forward pass and gradient approximations in the backward pass. Conceptually, this is the role STE plays: the model experiences quantization-like behavior while still remaining trainable.

So when people say “the forward pass is quantized but the backward pass is floating point,” they usually mean that the forward pass simulates quantization, while the backward pass uses an approximation such as STE to let gradients pass through.

4. Core Concepts of Quantization and QAT

4.1 PTQ vs. QAT

PTQ (Post-Training Quantization) converts a trained FP32 model into a lower-precision model, such as INT8, after training is finished.

QAT (Quantization-Aware Training) simulates quantization error during training so that the model can adapt to the effects of quantization before deployment.

In QAT, the model being trained is not yet a fully converted INT8 model. Instead, it is still a floating-point model that includes additional modules to simulate quantization behavior during the forward pass.

4.2 What Is an Observer?

An observer is a module that collects statistics from tensors, such as minimum and maximum values, in order to estimate quantization parameters like scale and zero-point.

The observer does not immediately convert tensors to INT8. Its role is to measure the value range and provide the information needed for quantization later.

- Observer: collects tensor statistics and estimates quantization parameters.
- Fake quantization: simulates rounding and clipping using those parameters.
- Convert step: creates the final quantized modules based on the collected information.

4.3 What Does `model_qat.train()` Actually Train?

Calling `model_qat.train()` does not mean that a fully converted INT8 model is being trained.

At this stage, `model_qat` is still a floating-point model with inserted observers and fake-quant modules. During training, the forward pass simulates quantization effects, but parameter updates are still performed in floating point.

- the forward pass includes simulated quantization behavior
- the model parameters are still trained in floating point
- the actual quantized model is created later by `convert(...)`

5. Meaning of `backend = "x86"`

Setting `backend = "x86"` means that the final quantized model is prepared and executed according to the x86 CPU quantization backend.

This does not mean that the INT8 data type itself is changed into some special x86-only numeric format. Instead, it affects implementation details such as default observer and fake-quant settings, weight packing format, and the low-level quantized kernels used during inference.

Common misunderstanding	Actual meaning
-------------------------	----------------

“Using the x86 backend changes the INT8 data type itself.”	No. INT8 remains INT8. What changes are the quantization rules, packed weight representation, and execution kernels.
“The backend refers to the training device.”	No. device and backend are different concepts. The device is where tensors and models are placed, while the backend controls quantized inference behavior.
“Changing the backend completely changes the model architecture.”	No. The high-level network structure stays the same, but converted module types, weight packing, numerics, and performance characteristics may differ.

6. Training on CUDA and Converting on CPU

It is possible to train the QAT model on CUDA when device is set to cuda.

In that case, QAT training runs on the GPU, but the final native INT8 conversion and inference are typically performed using a CPU backend such as x86.

This does not mean that the model moves back and forth between CPU and GPU at every training step. Instead, training data is moved from CPU to CUDA, the QAT model is trained on CUDA, and only after training a copy of the model is moved to CPU for conversion.

```
# training phase
images = images.to("cuda")
model_qat = model_qat.to("cuda")

# forward / backward / optimizer step on CUDA

# final conversion phase
model_qat_cpu = copy.deepcopy(model_qat).to("cpu")
model_int8 = torch.ao.quantization.convert(model_qat_cpu, inplace=False)

# int8 inference on CPU
```

7. Explanation of the Key Code Snippet

```
backend = "x86"
torch.backends.quantized.engine = backend

model_fp32.qconfig = torch.ao.quantization.get_default_qat_qconfig(backend)

model_qat = torch.ao.quantization.prepare_qat(
    model_fp32.train(),
    inplace=False
).to(device)

print(model_qat)
```

Code	Explanation
<code>backend = "x86"</code>	Defines the target quantization backend as x86 CPU.
<code>torch.backends.quantized.engine = backend</code>	Sets the quantized execution engine so that converted quantized operators use the x86 backend.
<code>get_default_qat_qconfig(backend)</code>	Loads the default QAT configuration for the selected backend, including observer and fake-quant settings.
<code>model_fp32.train()</code>	Switches the original floating-point model to training mode before QAT preparation.
<code>prepare_qat(...)</code>	Inserts observers and fake-quant modules into the model, producing a QAT-ready training model.
<code>.to(device)</code>	Moves the prepared QAT model to the selected device, such as CPU or CUDA.
<code>print(model_qat)</code>	Prints the prepared model so that inserted modules can be inspected.

This code is connected to the earlier concepts in a simple sequence: the backend defines which quantized inference rules and kernels are being targeted; the qconfig defines how quantization is simulated during training; `prepare_qat(...)` inserts the required observer and fake-quant modules; the model is trained as a floating-point model with simulated quantization effects; and later `convert(...)` creates the actual quantized model.

Using the x86 backend does not mean that INT8 itself becomes an x86-specific numeric type. The backend mainly affects the quantization configuration, observer and fake-quant defaults, packed internal representation of weights, and the low-level execution kernels used after conversion.

8. Why Is `model_fp32.train()` Used Before `prepare_qat(...)`?

The expression `model_qat = torch.ao.quantization.prepare_qat(model_fp32.train(), inplace=False).to(device)` can be understood as three connected steps.

```
model_fp32.train()
prepared = torch.ao.quantization.prepare_qat(model_fp32, inplace=False)
model_qat = prepared.to(device)
```

Here, `model_fp32.train()` does not mean that quantized training has already started. It simply puts the original floating-point model into training mode before QAT preparation.

This is usually done because QAT is a training-time workflow, and preparing the model in training mode is the standard and recommended pattern.

After that, `prepare_qat(...)` creates a new QAT-ready model by inserting observers and fake-quant modules.

The `.to(device)` call then moves that prepared QAT model to the selected device.

So the role of `model_fp32.train()` is mainly to ensure that QAT preparation starts from a model in training mode. It is part of the setup step, not the actual training loop.

9. Is `model_qat` in Training Mode by Default?

In this code flow, `model_qat` is typically created in training mode.

This happens because the original floating-point model is passed to `prepare_qat(...)` after calling `model_fp32.train()`.

So the QAT-prepared model usually starts in training mode as well.

However, it is still common to call `model_qat.train()` again inside the training loop. This is done for clarity and safety, especially because the model may have been switched to evaluation mode earlier using `model_qat.eval()`.

- `model_qat` is usually created in training mode
- `model_qat.eval()` switches it to evaluation mode
- `model_qat.train()` switches it back to training mode when needed

In other words, `model_qat` generally starts in training mode, but its mode is still managed explicitly during training and evaluation.

10. Summary of Commonly Confused Points

- Inheriting from `nn.Module` means that a class gains the basic functionality of a PyTorch model.
- `super().__init__()` runs the parent class initialization so that the child class can properly use inherited functionality.
- An observer does not directly convert tensors to INT8. It collects statistics needed for quantization.
- After `prepare_qat(...)`, `model_qat` is still not the final INT8 model. It is a floating-point QAT training model.
- `backend = "x86"` does not redefine the INT8 data type itself. It means that quantization rules and execution behavior are prepared for the x86 backend.
- CUDA is commonly used for QAT training, while native INT8 conversion and inference are usually performed with a CPU backend.
- `model_fp32.train()` before `prepare_qat(...)` is part of QAT preparation, while `model_qat.train()` and `model_qat.eval()` are used later to control the current runtime mode of the prepared QAT model.
- `model_qat` usually starts in training mode, but its mode is still switched explicitly during training and evaluation.